Cálculo de pi con los algoritmos Needles y Dartboard

Versiones seriales y paralelas con OpenMP

Reto 2

Presentado por:

Samuel Herrera Marin
Jonathan Gaviria Ocampo
Tomás Marulanda Aristizabal

Presentado a:

Ramiro Andrés Barrios Valencia

HPC: High Performance Computing
Ingeniería de Sistemas y Computación
2026

Tabla de contenido

Resumen 3
1 Introducción 3
2 Objetivos 3
3 Cómo funcionan los algoritmos 4
4 Revisión de las versiones seriales 5
5 Cómo repartimos el trabajo con OpenMP 5
6 Qué optimizamos en el procesador y en la memoria 6
7 Cómo medimos el desempeño 7
8 Pruebas para comprobar que el cálculo seguía bien 8
9 Resultados de la comparación de tiempos 9
10 Perfilado y medición de la memoria 11
11 Explicación de los resultados y sus límites 12
12 Conclusiones 12
13 Bibliografía y fuentes 13
Anexo A Código y comandos para repetir el trabajo 14
Anexo B Cinco repeticiones de las configuraciones más rápidas 14

Resumen

En este reto estudiamos dos maneras de calcular una aproximación de π usando muchos lanzamientos aleatorios: Dartboard y Needles. Primero revisamos que los programas originales funcionaran con un solo hilo. Después repartimos los lanzamientos entre varios hilos con OpenMP y comparamos los tiempos, el uso de memoria y las partes del programa que demandaban más trabajo.

La revisión incluyó 163 comprobaciones de funcionamiento y 990 mediciones de tiempo. Con diez millones de lanzamientos, el menor tiempo central de cinco ejecuciones fue de 5,646 milisegundos en Dartboard y 41,266 milisegundos en Needles. En ambos casos se usaron doce hilos. También comprobamos que guardar todos los datos aumentaba la memoria del proceso de unos 4,1 a 156,7 MiB. Estos resultados muestran que repartir el trabajo ayuda, pero que la cantidad de muestras, la forma de repartirlas y el almacenamiento también influyen.

1 Introducción

Los algoritmos de Monte Carlo utilizan experimentos aleatorios para aproximar una cantidad. En este caso, cada experimento consiste en lanzar un punto o una aguja. No necesitamos conocer el resultado del lanzamiento anterior para evaluar el siguiente. Por eso podemos repartir los lanzamientos entre varios hilos, es decir, varios grupos de trabajo dentro del programa.

OpenMP permite expresar ese reparto en C sin crear y administrar cada hilo manualmente. El propósito del reto fue comprobar si ambos algoritmos podían aprovecharlo y entender por qué algunas decisiones mejoran el tiempo mientras otras aportan poco o incluso lo aumentan. Realizamos las mediciones en un solo computador y usamos siempre los mismos datos para comparar de forma justa las versiones nuevas.

2 Objetivos

2.1 Objetivo general

Implementar y comparar versiones seriales y paralelas de Needles y Dartboard para estimar π, y explicar sus tiempos y su uso de memoria a partir de pruebas y mediciones.

2.2 Objetivos específicos

• Explicar cómo calcula π cada algoritmo y qué condiciones debe cumplir.

• Comprobar que los programas originales se ejecutan con un solo hilo y producen resultados coherentes.

• Repartir los lanzamientos con OpenMP sin perder aciertos ni cambiar las muestras.

• Comparar diferentes cantidades de hilos, formas de reparto y opciones de compilación.

• Medir el tiempo total del cálculo, el tiempo de sus partes y la memoria utilizada.

• Relacionar las diferencias observadas con las decisiones tomadas en el programa.

3 Cómo funcionan los algoritmos

3.1 Dartboard o lanzamiento de puntos

Podemos imaginar un círculo de radio uno dentro de un cuadrado de lado dos. El programa lanza puntos al azar sobre el cuadrado y cuenta cuántos quedan dentro del círculo. El área del círculo es π y la del cuadrado es cuatro. Por eso, la proporción de puntos que cae dentro del círculo se acerca a $\pi/4$ cuando se realizan muchos lanzamientos.

$$\pi \text{ estimado} = 4 \times \text{puntos dentro del círculo} / \text{total de puntos}$$

Para saber si un punto está dentro, usamos $x^2 + y^2 \leq 1$. Esta comparación evita calcular una raíz cuadrada y ya estaba presente en la versión original. El algoritmo solo necesita conservar las coordenadas del punto actual y el conteo acumulado.

3.2 Needles o aguja de Buffon

En Needles imaginamos una superficie con líneas paralelas y lanzamos agujas sobre ella. En cada lanzamiento generamos la distancia del centro de la aguja a la línea más cercana y el ángulo de la aguja. Con esos dos datos sabemos si alcanza a cruzar una línea.

Llamamos L a la longitud de la aguja y D a la separación entre líneas. La fórmula usada requiere que la aguja no sea más larga que esa separación. Si d es la distancia y θ el ángulo, la condición de cruce es $d \leq (L/2) \times \sin(\theta)$. A partir de la proporción de cruces obtenemos la aproximación:

$$\pi \text{ estimado} = 2 \times L \times \text{total de agujas} / (D \times \text{agujas que cruzan})$$

En nuestro programa L=D=1, por lo que la expresión queda en 2 × total de agujas / cruces. Si ninguna aguja cruza, no podemos dividir entre cero y el programa indica que no hay estimación. Needles realiza un cálculo de seno en cada lanzamiento, lo cual lo hace más costoso que Dartboard.

Esta versión de Needles utiliza una constante π conocida para generar el ángulo. Seguimos esa formulación del programa original como ejercicio de simulación; no se presenta como una manera independiente de descubrir nuevos dígitos de π.

3.3 Qué significa obtener una aproximación

Ninguno de los dos métodos garantiza los mismos decimales en cada ejecución. El azar hace que los resultados cambien si cambiamos las muestras. En general, usar cien veces más lanzamientos reduce unas diez veces el error típico, aunque una ejecución concreta pueda quedar más cerca o más lejos del valor real.

Con la misma cantidad de muestras, Dartboard tiene menor variación esperada que Needles para L=D. Según la comparación matemática, Needles necesita aproximadamente 2,09 veces las muestras para alcanzar una precisión típica parecida. Usar más hilos con la misma cantidad de lanzamientos mejora el tiempo; no mejora por sí solo la precisión.

4 Revisión de las versiones seriales

4.1 Qué comprobamos antes de paralelizar

Revisamos Needle.c y Dartboard.c y los compilamos nuevamente desde sus fuentes. Ambos realizan el cálculo en un único ciclo y no incluyen instrucciones OpenMP ni creación de hilos. Los compilamos sin activar OpenMP y no aparecieron advertencias. Conservamos estos archivos sin cambios como punto de partida del trabajo.

Los originales hacen un millón de lanzamientos y toman la hora como punto de partida para generar los números aleatorios. Por eso sus respuestas pueden cambiar al ejecutarlos otra vez. Las fórmulas revisadas corresponden a las condiciones explicadas en la sección anterior.

Algoritmo	Aciertos o cruces	π estimado	Tiempo impreso (s)
Dartboard	785356	3,1414240000	0,024
Needles	636223	3,1435518678	0,063

Tabla 1. Resultados de los originales con un millón de lanzamientos. Fuente: verificacion.json.

Ambas aproximaciones quedaron dentro del margen de variación aceptado por la prueba. No esperábamos una cantidad fija de decimales correctos. Comprobar una ejecución tampoco demuestra por sí solo que un generador de números aleatorios sea adecuado para cualquier estudio.

4.2 Una base serial para comparar tiempos

Además de los originales, montecarlo.c tiene un modo serial nuevo. Hace el mismo trabajo que la versión paralela, con el mismo generador y la misma semilla, pero sin abrir una región de trabajo entre varios hilos. La semilla es el valor inicial que permite repetir las mismas muestras. Esta base es necesaria porque comparar directamente el original con la versión nueva mezclaría el efecto del paralelismo con el cambio de generador.

5 Cómo repartimos el trabajo con OpenMP

5.1 Las mismas muestras con cualquier cantidad de hilos

La función uniform01 calcula dos valores para cada lanzamiento a partir de su número y de una semilla común. El lanzamiento cero usa las posiciones cero y uno; el siguiente usa dos y tres, y así sucesivamente. Empleamos una mezcla de enteros basada en SplitMix64 [1].

La ventaja práctica es que cada hilo puede generar sus muestras sin esperar al anterior ni modificar el estado de otro hilo. Si un lanzamiento cambia de hilo, conserva sus mismos valores. Esto nos permite comprobar que una diferencia de tiempo no se debe a haber cambiado las muestras. Las pruebas del reto no sustituyen una evaluación especializada de la calidad del generador.

5.2 Cada hilo cuenta sus propios aciertos

Usamos parallel for para repartir las vueltas del ciclo. Cada hilo evalúa los lanzamientos que recibe y lleva un conteo propio. Al terminar, reduction(+:hits) suma esos conteos [2]. Por ejemplo, si tres hilos obtienen 3, 2 y 4 aciertos, el total final es 9.

```
#pragma omp parallel for default(none) shared(n, seed) \
        schedule(runtime) reduction(+:hits)
```

La reducción evita que todos intenten cambiar el mismo contador en cada lanzamiento. Esa actualización simultánea podría perder aciertos si no se protege. Otra solución sería obligar a los hilos a turnarse cada vez que suman, pero eso añadiría muchas esperas. Con la reducción solo hay que combinar los resultados al final. Las variables del punto o de la aguja se usan de forma local, mientras N y la semilla solo se consultan.

5.3 Formas de entregar los lanzamientos a los hilos

Opción	Cómo reparte el trabajo
static	Entrega bloques definidos de iteraciones a los hilos.
dynamic	Cada hilo solicita otro bloque de 1024 muestras cuando termina el suyo.
guided	Empieza con bloques grandes y los reduce; el mínimo usado fue 1024.

Tabla 2. Formas de reparto evaluadas con schedule(runtime) [3].

static suele ser una buena primera opción si todos avanzan a un ritmo parecido. dynamic y guided permiten que un hilo que termina antes tome más trabajo. Esa flexibilidad tiene un costo de organización, por lo que decidimos medir las tres alternativas en lugar de suponer cuál ganaría. En esta campaña dynamic fue mejor con doce hilos, aunque eso puede cambiar en otro equipo.

Solicitamos 1, 2, 4, 6 o 12 hilos y registramos cuántos participaron realmente. Desactivamos el cambio automático del tamaño del equipo. Esto es distinto del reparto dynamic, que decide qué bloque recibe cada hilo, no cuántos hilos hay.

6 Qué optimizamos en el procesador y en la memoria

6.1 Opciones del compilador

El compilador transforma el código C en un programa ejecutable. Sus opciones pueden cambiar la forma de hacer los cálculos sin cambiar lo que buscamos obtener. Probamos tres alternativas para ver si una mejoraba los tiempos [5].

Nombre	Opciones	Qué buscamos
O2	-O2	Una versión optimizada de uso general.
O3	-O3	Permitir más mejoras automáticas del compilador.
native	-O3 -march=native	Aprovechar las instrucciones disponibles en este procesador.

Tabla 3. Compilaciones usadas en la comparación de tiempos.

También simplificamos la condición de Needles para L=D=1, cancelando los factores 1/2 de ambos lados. Conservamos la prueba de Dartboard sin raíz cuadrada y usamos funciones pequeñas para facilitar el trabajo del compilador. No medimos por separado cada simplificación, por lo que no le asignamos un porcentaje individual de mejora.

6.2 Prueba de varias operaciones por instrucción

Probamos SIMD, una técnica que permite que una instrucción del procesador trabaje con varios datos a la vez. Añadimos parallel for simd y comparamos sus tiempos con los del reparto normal [4]. También revisamos los mensajes del compilador para comprobar qué partes logró transformar.

El resultado fue que SIMD no dio una mejora constante. Escribir la instrucción en el código no asegura que todo el cálculo pueda aprovecharla. Por eso la presentamos como una técnica evaluada y no como una mejora demostrada para ambos algoritmos.

6.3 Generar y usar cada muestra de inmediato

La versión normal genera un punto o una aguja, lo evalúa y continúa con el siguiente. Solo conserva los datos actuales y el conteo. Así no necesita guardar millones de muestras ni reservar memoria dentro del ciclo.

Para medir el efecto creamos buffered, una versión de comparación que primero guarda todos los valores en dos arreglos y después los evalúa. Cada arreglo contiene N números de ocho bytes, de modo que ambos requieren 16N bytes. Con diez millones de muestras son unos 152,59 MiB adicionales.

Los programas originales ya evitaban guardar todas las muestras. Mantuvimos esa buena decisión y agregamos la versión con arreglos para comprobar su costo. En la versión paralela, cada hilo necesita sus datos locales y su conteo, pero la memoria no crece con cada nuevo lanzamiento.

7 Cómo medimos el desempeño

7.1 Equipo utilizado

Componente	Valor registrado
Procesador	Intel(R) Core(TM) i5-10400 CPU @ 2.90GHz
Procesadores lógicos	12
Sistema operativo	Windows 10, versión 19045, 64 bits
Compilador	GCC 16.1.0 de MSYS2 / MinGW-w64
Python	3.11.1
Fecha de las mediciones	4 de noviembre de 2026

Tabla 4. Entorno de las pruebas. Fuente: entorno.json.

Las mediciones corresponden a este computador. No fijamos un núcleo específico para cada hilo ni controlamos la temperatura, la frecuencia efectiva o todas las tareas de Windows. Por eso los tiempos pueden variar y no deben tomarse como una promesa de desempeño para cualquier equipo.

7.2 Benchmark o comparación repetida de tiempos

El benchmark consistió en ejecutar las variantes varias veces y comparar cuánto tardaban. Usamos 100 mil, un millón y diez millones de lanzamientos. Para cada algoritmo, tamaño y

compilación probamos once configuraciones: serial, buffered, OpenMP static con cinco cantidades de hilos, SIMD static con uno y doce hilos, y OpenMP con doce hilos usando dynamic y guided.

3 compilaciones × 2 algoritmos × 3 tamaños × 11 configuraciones × 5 repeticiones

Total de la campaña: 990 mediciones

El script benchmark.py ejecutó los procesos uno por uno en un orden mezclado con una semilla fija. Antes de medir hicimos calentamientos. Todas las repeticiones usaron la semilla 42 para conservar los mismos lanzamientos. Así medimos la variación de tiempo con un trabajo igual en cada caso.

7.3 Qué tiempo y qué memoria registramos

Medimos por separado el cálculo y la ejecución completa del proceso. El primer tiempo incluye generar las muestras, evaluarlas y sumar los conteos, después de preparar el equipo de hilos. No incluye abrir el proceso ni imprimir la respuesta. El segundo sí incluye toda la ejecución, y permite ver si el arranque pesa demasiado cuando la simulación es pequeña.

En Windows usamos QueryPerformanceCounter, un reloj de alta resolución [6]. Descartamos una campaña inicial porque el reloj de OpenMP devolvió algunos tiempos cero para trabajos pequeños, y repetimos las mediciones con el nuevo reloj. También registramos el tiempo CPU del proceso, aunque su resolución no alcanzó para distinguir bien algunos casos cortos.

Para cada configuración publicamos la mediana de cinco tiempos: el valor que queda en la mitad después de ordenarlos. Conservamos también el menor, el mayor y una medida de su dispersión. La aceleración, llamada speedup, indica cuántas veces se reduce el tiempo respecto del serial del mismo programa. La eficiencia divide esa aceleración entre los hilos utilizados.

Aceleración = tiempo serial / tiempo paralelo

La memoria registrada es el máximo uso de memoria residente del proceso completo, que incluye el programa y sus bibliotecas. Medimos también cuántos bytes ocupan únicamente los arreglos. No medimos los fallos de caché ni la velocidad de transferencia de memoria del hardware.

8 Pruebas para comprobar que el cálculo seguía bien

8.1 Las 163 comprobaciones

Prueba	Casos	Qué revisamos
Referencia en Python	108	Mismos conteos que una implementación separada de las fórmulas.
Cambios de reparto	36	Mismo conteo con static, dynamic y guided.
Datos de entrada incorrectos	7	Rechazo de tamaños, hilos o valores fuera del rango permitido.
Varias semillas	10	Aproximaciones coherentes con el error aleatorio esperado.
Programas originales	2	Compilación, ejecución y revisión de sus aproximaciones.
Total	163	Todas las comprobaciones pasaron.

Tabla 5. Pruebas de verify.py. Evidencia conservada en verificacion.json.

Para la referencia usamos 1, 17 y 10003 lanzamientos, con semillas distintas. Comparamos los modos serial, buffered, OpenMP con uno, dos y cuatro hilos y SIMD con cuatro hilos, en las tres compilaciones. En las pruebas de reparto usamos 100003 lanzamientos y cuatro hilos. Esos tamaños permiten comprobar también qué ocurre cuando la cantidad no se divide exactamente entre los hilos.

Los siete casos incorrectos incluyeron cero o un número negativo de muestras, cero hilos, un algoritmo inexistente, pedir varios hilos en serial y valores enteros demasiado grandes. Para las pruebas con varias semillas usamos un millón de muestras y un margen amplio de seis errores estándar, que tiene en cuenta la variación esperada por el azar. Eso comprueba coherencia; no exige decimales idénticos entre semillas.

8.2 Ejemplo con diez millones de muestras

Algoritmo	Aciertos o cruces	π estimado	Error absoluto
Dartboard	7855863	3,142345200000	0,0007525464
Needles	6368382	3,140515126134	0,0010775275

Tabla 6. Semilla 42. Estos conteos coinciden entre los modos comparados.

Además de las 163 comprobaciones, el benchmark revisó durante sus 990 ejecuciones que el conteo no cambiara para un mismo algoritmo, tamaño y semilla. Esta igualdad exacta era importante: una versión más rápida no sería una mejora si perdiera lanzamientos o sumara mal.

9 Resultados de la comparación de tiempos

9.1 Efecto de las opciones de compilación

Compilación	Dartboard en ms	Needles en ms
O2	50,623	433,635
O3	37,679	420,053
native	39,844	419,194

Tabla 7. Versiones seriales con diez millones de muestras. Cada valor es la mediana de cinco ejecuciones.

O3 redujo el tiempo frente a O2 aproximadamente 1,34 veces en Dartboard y 1,03 veces en Needles. La opción native no fue mejor de forma constante. Cuando las diferencias son pequeñas y los tiempos varían, conviene evitar conclusiones demasiado firmes sobre una opción del compilador.

9.2 Efecto de repartir entre más hilos

La siguiente tabla usa la compilación native y diez millones de muestras. Los tiempos están en milisegundos, es decir, milésimas de segundo. Cada aceleración se calcula frente al serial de ese mismo algoritmo.

Configuración	Dartboard ms	Aceleración	Needles ms	Aceleración
Serial	39,844	1,00×	419,194	1,00×
2 hilos static	17,365	2,29×	207,579	2,02×

Configuración	Dartboard ms	Aceleración	Needles ms	Aceleración
4 hilos static	12,571	3,17×	110,113	3,81×
6 hilos static	9,989	3,99×	77,737	5,39×
12 hilos static	7,805	5,11×	67,459	6,21×
12 hilos dynamic	5,646	7,06×	44,258	9,47×
12 hilos guided	5,837	6,83×	46,018	9,11×

Tabla 8. Comparación con igual compilación, tamaño y semilla.

Ambos algoritmos redujeron su tiempo con más hilos. Needles obtuvo mayor aceleración relativa porque tiene más trabajo por lanzamiento y puede repartirlo entre los hilos. Aun así, Dartboard terminó mucho antes en tiempo total. Pasar de seis a doce hilos tampoco duplicó la mejora: hay organización del trabajo, esperas y recursos compartidos.



Figura 1. Aceleración con diez millones de muestras y compilación native. La línea discontinua representa la mejora ideal proporcional a los hilos. Fuente: resumen.csv.

9.3 El menor tiempo y la mayor aceleración pueden ser distintos

Algoritmo	Compilación	Serial ms	Paralelo ms	Aceleración
Dartboard	O2	50,623	5,778	8,76×
Dartboard	native	39,844	5,646	7,06×
Needles	O3	420,053	41,266	10,18×
Needles	native	419,194	44,258	9,47×

Tabla 9. Diez millones de muestras y doce hilos con dynamic. Cada fila tiene su propia referencia serial.

Dartboard native tuvo el menor tiempo observado: 5,646 ms, con una aceleración de 7,06 veces. Dartboard O2 tuvo una aceleración mayor, 8,76 veces, porque su versión serial tardaba más. Por eso una cifra mayor de speedup no siempre significa un menor tiempo final.

En Needles, O3 obtuvo el menor tiempo observado: 41,266 ms frente a 420,053 ms de su serial. La aceleración fue de 10,18 veces y la eficiencia de 84,8 % al dividirla entre doce hilos. Estas son las mejores medianas encontradas en la campaña, no resultados garantizados para otra ejecución o computador.

9.4 Por qué también medimos la ejecución completa

Con 100 mil muestras, Dartboard serial native tardó 0,327 ms en el cálculo, pero 10,550 ms como proceso completo. Con doce hilos y static, el cálculo bajó a 0,137 ms y el proceso completo tardó 12,387 ms. En ese caso, abrir y preparar el trabajo pesa más que lo ahorrado en el cálculo. Por eso OpenMP puede ser útil en simulaciones grandes y no compensar en una ejecución pequeña.

10 Perfilado y medición de la memoria

10.1 Qué parte del programa tardó más

El benchmark nos dice cuánto tarda cada versión. El perfilado busca explicar en qué parte se va ese tiempo. Primero intentamos usar gprof, una herramienta que registra el trabajo de las funciones. Compilamos la versión preparada para esa herramienta y la ejecutamos, pero en este entorno sus archivos no contenían muestras útiles.

Para obtener evidencia real, medimos por separado dos partes de buffered: generar y guardar las muestras, y leerlas para evaluar si acertaban. Usamos treinta millones de lanzamientos, descartamos un calentamiento y tomamos cinco ejecuciones por algoritmo. Esta forma de medir se llama perfilado por instrumentación: colocar relojes alrededor de las partes que queremos estudiar.

Algoritmo	Generar y guardar ms	Leer y evaluar ms	Evaluación respecto de ambas fases
Dartboard	145,288	40,907	22,0 %
Needles	139,308	1123,755	89,0 %

Tabla 10. Perfilado de buffered con treinta millones de muestras. Fuente: perfil_fases.csv y perfil_estado.json.

En Dartboard tardó más generar y guardar los datos. En Needles la evaluación representó cerca del 89 % de la suma de esas dos fases. Es coherente con el cálculo de seno por lanzamiento, pero no significa que hayamos medido un 89 % exclusivamente dentro de sin: la fase también incluye lecturas y el conteo. Estos porcentajes corresponden a buffered y no se trasladan directamente a la versión que no guarda arreglos.

10.2 Qué mostró la prueba de memoria

Algoritmo	Versión	Tiempo ms	Pico de memoria MiB
Dartboard	Sin arreglos	39,844	4,12
Dartboard	Con arreglos	73,709	156,69
Needles	Sin arreglos	419,194	4,12
Needles	Con arreglos	443,336	156,69

Tabla 11. Diez millones de muestras, compilación native y un hilo. Pico de memoria del proceso completo.

Guardar las muestras elevó la memoria de unos 4,1 a 156,7 MiB. La diferencia concuerda con los dos arreglos de 152,59 MiB adicionales. La versión sin arreglos fue cerca de 1,85 veces más rápida en Dartboard y 1,06 veces en Needles. En Needles el cálculo geométrico pesa más, de modo que el almacenamiento representa una parte menor del tiempo.



Figura 2. Comparación de memoria y tiempos por partes del programa. El gráfico de memoria usa diez millones de muestras; el perfilado usa treinta millones. Fuente: mediciones del proyecto.

10.3 Qué pasó con SIMD

Algoritmo	OpenMP normal ms	OpenMP con SIMD ms
Dartboard	7,805	8,402
Needles	67,459	63,671

Tabla 12. Doce hilos, static, native y diez millones de muestras.

Dartboard no mejoró con SIMD en esta comparación. En Needles la diferencia fue pequeña frente a la variación de los tiempos. Los mensajes del compilador también mostraron que el ciclo completo de muestras no se transformó de forma efectiva en operaciones vectoriales. Por eso no presentamos SIMD como una mejora confirmada en este trabajo.

11 Explicación de los resultados y sus límites

11.1 Repartir más trabajo puede aprovechar mejor los hilos

Needles tarda más por lanzamiento. Cuando repartimos muchos lanzamientos, los hilos tienen suficiente trabajo útil para compensar parte del costo de organización. Esto ayuda a entender su mayor aceleración relativa. Dartboard hace menos trabajo por muestra y termina antes, aunque su aceleración frente al serial sea menor.

11.2 La forma de reparto y Windows también influyen

Con doce hilos, Needles native y static tuvo tiempos entre 44,065 y 68,801 ms; con dynamic, entre 39,999 y 44,676 ms. El reparto flexible pudo compensar hilos que avanzaron más lentamente, pero no medimos por separado todas las causas. El sistema operativo y otras tareas también pueden alterar el ritmo de ejecución.

En algunos casos de pocos hilos, la eficiencia calculada superó el 100 %. Eso no demuestra que cada hilo rindiera más de su capacidad física. La versión serial y la región OpenMP se compilan por caminos diferentes y también hay variación temporal. Por ejemplo, Dartboard con OpenMP de un hilo ya fue más rápido que su serial.

11.3 Qué podemos afirmar con estas pruebas

Las pruebas respaldan que los conteos se conservaron en los casos revisados, que OpenMP redujo los tiempos de las simulaciones grandes y que guardar las muestras aumentó la memoria y el tiempo. No bastan para afirmar que una compilación o una forma de reparto siempre será superior. Tampoco medimos por separado el costo de cada instrucción ni los efectos internos de la caché.

Para profundizar sería útil repetir en otros equipos, asignar los hilos de forma controlada a los núcleos y medir en condiciones similares de energía y carga. También convendría usar una herramienta de perfilado de funciones que sí produzca datos útiles en el entorno y evaluar el generador con pruebas especializadas.

12 Conclusiones

• Los programas originales son seriales y sus fórmulas son correctas para las condiciones usadas. Esto permitió continuar desde una base revisada.

• OpenMP permitió repartir los lanzamientos y sumar los conteos sin cambiar el resultado. Pasaron las 163 comprobaciones y los controles de las 990 mediciones.

• Con diez millones de muestras, los menores tiempos observados fueron 5,646 ms en Dartboard native y 41,266 ms en Needles O3, con doce hilos y dynamic.

• El mayor speedup de Dartboard fue 8,76 con O2, mientras native tuvo el menor tiempo y un speedup de 7,06. En Needles O3, el speedup fue 10,18.

• O3 ayudó especialmente a Dartboard. Las opciones native y SIMD no mejoraron siempre, de modo que fue necesario probarlas y no darlas por superiores.

• Generar y usar cada muestra de inmediato mantuvo bajo el uso de memoria. Guardarlas todas elevó el pico de unos 4,1 a 156,7 MiB y aumentó el tiempo.

• El perfilado por fases mostró un peso mayor de la evaluación en Needles. gprof no produjo datos útiles y esa limitación se conservó en el informe.

• En estas condiciones, Dartboard fue más rápido y tiene menor variación esperada por muestra. Las conclusiones deben revisarse al cambiar el equipo o las condiciones de ejecución.

13 Bibliografía y fuentes

[1] Steele, G. L., Lea, D. y Flood, C. H. (2014). Fast Splittable Pseudorandom Number Generators. OOPSLA. DOI 10.1145/2660193.2660195.
https://gee.cs.oswego.edu/dl/papers/oopsla14.pdf

[2] OpenMP Architecture Review Board. (2021). OpenMP API Specification 5.2. Reduction clause.
https://www.openmp.org/spec-html/5.2/openmpsu52.html

[3] OpenMP Architecture Review Board. OpenMP API 5.2 Reference Guide. Planificación y funciones del runtime.
https://www.openmp.org/wp-content/uploads/OpenMPRefGuide-5.2-Web-2024.pdf

[4] OpenMP Architecture Review Board. (2021). OpenMP API Specification 5.2. SIMD construct.
https://www.openmp.org/spec-html/5.2/openmpse60.html

[5] GNU Project. Using the GNU Compiler Collection. Optimize Options y Developer Options.
https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html

[6] Microsoft. Acquiring high-resolution time stamps. Documentación de Windows.
https://learn.microsoft.com/en-us/windows/win32/sysinfo/acquiring-high-resolution-time-stamps

[7] Archivos del proyecto y resultados experimentales del 4 de noviembre de 2026. Fuentes: Needle.c, Dartboard.c y montecarlo.c. Scripts: build.py, verify.py, benchmark.py, profile.py y graficas.py. Resultados: mediciones.csv, resumen.csv, verificacion.json, entorno.json, perfil_fases.csv, perfil_estado.json y vector_*.txt.

Anexo A Código y comandos para repetir el trabajo

A1 Cálculo serial de los programas originales

Estos fragmentos muestran las operaciones centrales. En Dartboard contamos puntos dentro del círculo; en Needles contamos agujas que cruzan. Los archivos originales completos también incluyen la preparación y la impresión de resultados.

```
/* Dartboard */
for (long long i = 0; i < N; i++) {
    x = 2.0 * rand() / RAND_MAX - 1.0;
    y = 2.0 * rand() / RAND_MAX - 1.0;
    if (x * x + y * y <= 1.0) dentro++;
}
pi = 4.0 * (double)dentro / N;
```

```
/* Needles, con L=D=1 */
for (long long i = 0; i < N; i++) {
    distancia = ((double)rand() / RAND_MAX) * (D / 2.0);
    angulo = ((double)rand() / RAND_MAX) * (PI / 2.0);
    if (distancia <= (L / 2.0) * sin(angulo)) cruces++;
}
if (cruces > 0) pi = (2.0 * L * N) / (D * cruces);
```

A2 Cómo repetir las pruebas

```
cd "E:\HPC\Reto 2"
python build.py
python verify.py
python profile.py
python benchmark.py --repeats 5
python graficas.py
```

El perfilado y el benchmark se ejecutan por separado. Los scripts usan Python y GCC con OpenMP; graficas.py necesita matplotlib. Ejemplos de ejecución directa para diez millones de muestras y semilla 42:

```
.\build\mc_native.exe dartboard serial 10000000 1 42 static
.\build\mc_native.exe dartboard omp 10000000 12 42 dynamic
.\build\mc_O3.exe needles serial 10000000 1 42 static
.\build\mc_O3.exe needles omp 10000000 12 42 dynamic
```

A3 Opciones técnicas que se mantuvieron iguales

Las compilaciones nuevas usan -fopenmp para habilitar OpenMP, -ffp-contract=off para evitar ciertas diferencias por operaciones aritméticas fusionadas y -fno-math-errno porque el programa no consulta ese indicador después de sin. No se activó -ffast-math. La compilación para intentar gprof usó -O2 -g -pg -fno-inline y no se mezcló con los tiempos del benchmark. Los comandos completos se conservan en compilacion.txt.

Anexo B Cinco repeticiones de las configuraciones más rápidas

Estas son las cinco mediciones de las configuraciones con menor tiempo central para diez millones de muestras. Los archivos conservan las 990 ejecuciones completas; no elegimos una sola repetición favorable para publicar el resultado.

Dartboard native

Repetición	Cálculo ms	Aciertos o cruces	Proceso completo ms
1	5,6065	7855863	16,9555
2	5,6458	7855863	15,5800
3	5,1804	7855863	15,4716
4	5,8367	7855863	18,8652
5	6,1600	7855863	18,8204
Mediana	5,646	Mismo conteo	16,9555

Tabla 13. Dartboard native, doce hilos y dynamic. Fuente: mediciones.csv.

Needles O3

Repetición	Cálculo ms	Aciertos o cruces	Proceso completo ms
1	40,3426	6368382	50,4879
2	41,2657	6368382	53,2515
3	44,6454	6368382	55,0839
4	40,7567	6368382	52,3297
5	45,3694	6368382	55,0665
Mediana	41,266	Mismo conteo	53,2515

Tabla 14. Needles O3, doce hilos y dynamic. Fuente: mediciones.csv.